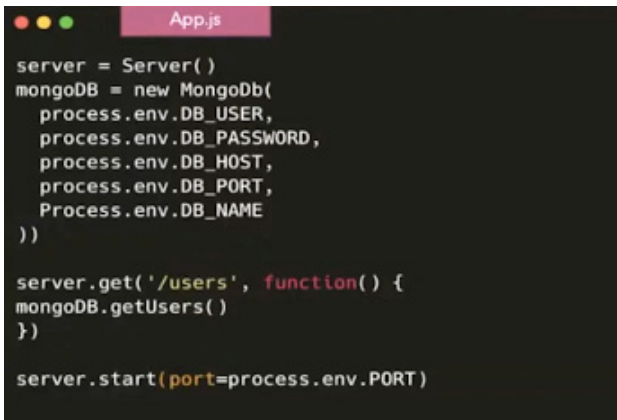




```

server = Server()
mongoDB = new MongoClient(
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  process.env.DB_HOST,
  process.env.DB_PORT,
  {
    useNewUrlParser: true,
    useUnifiedTopology: true
  }
)

server.get('/users', function() {
  mongoDB.getUsers()
})

server.start(port=process.env.PORT)

```

Figure 8: Your app will now be secure



```

DB_NAME=test_db
DB_USER=username
DB_PASSWORD=password
DB_DOMAIN=localhost
DB_PORT=3000
HOST=http://localhost

```

Figure 9: Dotenv development file

writing any of the credentials in these files, but just the path to the secrets manager. An example of code using Node.js is shown in Figure 8.

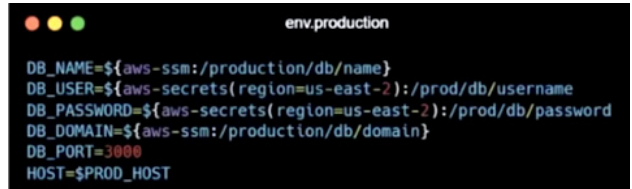
We are just accessing the *process* dotenv file directly without writing code to fetch any of the environment variables inside the code. However, the command to run the app needs to change.

```
secretsfoundry run --stage="development" -- script "node app.js"
```

Behind the code, it looks at the environment variables. For example, if you see the *dotenv development* file, there is a database name, user name and password in plain text. It is safe in the local development environment and developers can access it using the *dotenv development.local* command. SecretsFoundry will read each of the variables and will put these as environment variables to run the application process. This may just be simple environment file parsing, but more critical changes happen in the production file.

```
DB_NAME=${aws-ssm:/production/db/name}
```

In the *dotenv production* file, the database name is written as *aws-ssm*, followed by the path to the secret. It tells your SecretsFoundry to fetch the value from the AWS Parameter Store with this key — *production/DB/name*, so as to give the actual value.



```

DB_NAME=${aws-ssm:/production/db/name}
DB_USER=${aws-secrets(region=us-east-2):/prod/db/username}
DB_PASSWORD=${aws-secrets(region=us-east-2):/prod/db/password}
DB_DOMAIN=${aws-ssm:/production/db/domain}
DB_PORT=3000
HOST=$PROD_HOST

```

Figure 10: Dotenv production file

```
DB_USER=${aws-secrets(region=us-east-2):/prod/db/username}
```

In the DB user, the code should be written in a way that it can be fetched from the AWS Secrets Manager. In a similar way, we integrate with different providers like HashiCorp Vault and Amazon S3, so that you can just write in the *env* file where the source of the credential is, and SecretsFoundry will fetch it dynamically at runtime. This way it is much safer to commit the *dotenv production* file to Git.

How does SecretsFoundry work?

SecretsFoundry reads every variable defined in the *dotenv.** file according to the stage sent by the user in the *run* command and loads the necessary values.

Here is an example:

```
DB_PASSWORD=${aws-ssm:/app/staging/DB_PASSWORD}
```

You can see that the format of defining each environment (everything under the dollar symbol) is a variable that SecretsFoundry will try to parse out and first see the provider. The AWS Parameter Store is the provider here, with */app/staging/DB_PASSWORD* being the path to the secret inside it.

SecretsFoundry uses AWS APIs and secrets manager APIs to get values, and then puts in the environment variable to run the code. So your code will find the value of */app/staging/DB_PASSWORD* automatically in the environment variable *DB_password*.

This way, a developer can have as many *dotenv* files inside your repository and use the corresponding environment file by just passing this as an argument.

At the moment, SecretsFoundry can integrate only with AWS Parameter Store, AWS Secrets Manager, AWS S3 and HashiCorp Vault. It may soon be compatible with GCP Secrets Engine and Azure Key Vault. SecretsFoundry is currently available as an open source NPM package.

Container support

SecretsFoundry can also support containers and Kubernetes.

```
ENTRYPOINT ["secretsfoundry", "run", "--stage", "prod", "-s", "node example.js"]
```